High-Level Architecture Document

Face Recognition-Based Smart Attendance Management System

Project ID: 20

Version 1.0

UE24CS341A - Software Engineering, Jackfruit Mini-Project

Team Details

Field	Details
Team Name	SE GROUP
Team Member 1	MITHUN S | SRN: PES2UG24AM208
Team Member 2	BHARATH H G | SRN: PES2UG24AM197
Team Member 3	VARUN C H | SRN: PES2UG24AM180
Team Member 4	YASHWANT ADITYA VARMA | SRN: PES2UG24AM190
Prepared by	MITHUN S
Date Created	5.10.2026

1. Introduction

1.1 Purpose

This document describes the high-level architecture of the Face Recognition-Based Smart Attendance Management System with Analytics and Reporting (Project ID 20). It supplements the Software Requirements Specification (SRS, Phase 1) by showing how the system is structured into layers and components, how those components interact, and the technology choices behind each one. It is intended to guide the team's detailed design (SW Design Document) and implementation.

1.2 Scope

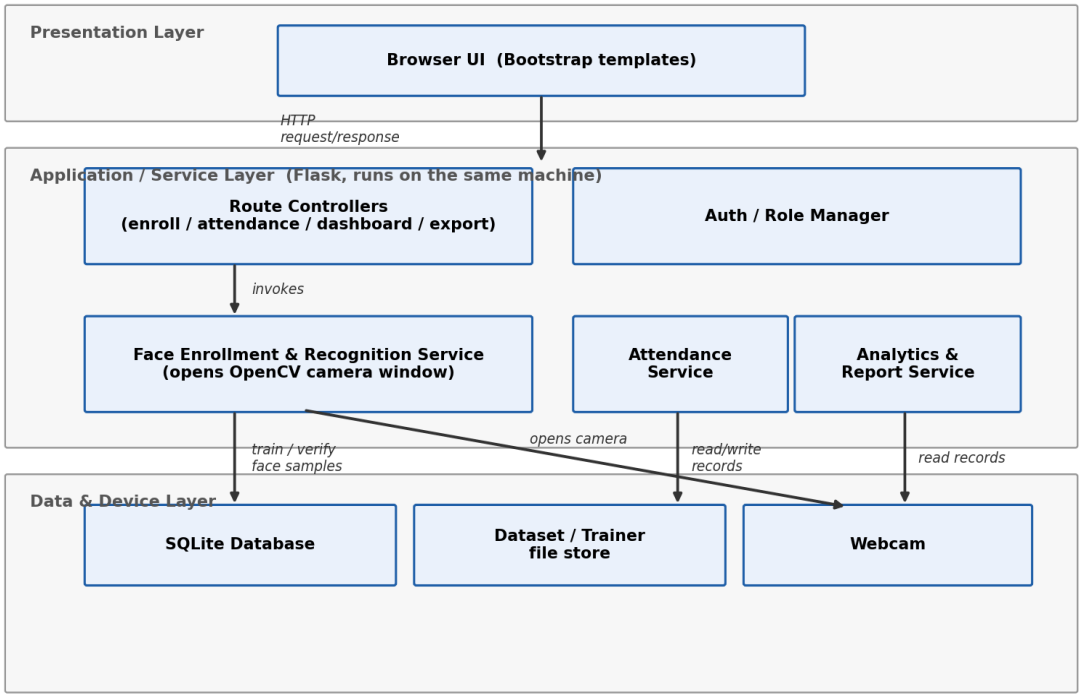
This document covers the architecture of the system as implemented: a single Flask web application, an OpenCV-based face enrollment/recognition module, and a SQLite database, all running on one machine (the faculty/admin's laptop). It does not cover multi-server deployment, load balancing, or cloud hosting, which are out of scope for this mini-project (see SRS Section 1.3, Product Scope).

2. Architectural Style and Rationale

The system follows a simple 3-layer architecture: a Presentation Layer (browser-based UI), an Application/Service Layer (Flask routes and service modules), and a Data & Device Layer (SQLite database, local file storage for face data, and the webcam). This style was chosen, rather than a more elaborate microservices or client-server-API split, because:

- The team is a small group of 3-4 students with a tight timeline - a monolithic Flask app is faster to build, run, and debug than multiple deployed services.
- It still enforces separation of concerns (routes vs. business logic vs. data access), satisfying the course's requirement for modular, maintainable, extensible design (see SRS Section 2.5) without the operational overhead of real microservices.
- The webcam and recognition logic must run on the same physical machine as the browser session (a remote server cannot see a local camera), which naturally rules out a fully decoupled client-server split for this feature.

3. High-Level Component Diagram



The diagram shows three layers. The Presentation Layer is the browser-rendered UI (Bootstrap templates served by Flask). The Application/Service Layer holds the Flask route controllers plus four service modules: Face Enrollment &

Recognition Service, Attendance Service, Analytics & Report Service, and Auth/Role Manager. The Data & Device Layer holds the SQLite database, the dataset/trainer file store (captured face images and the trained recognition model), and the webcam itself.

4. Component Responsibilities

Component	Responsibility	Key Technology
Browser UI	Renders forms and dashboards; submits HTTP requests for enroll/attendance/dashboard actions.	HTML5, Bootstrap 5
Route Controllers	Receives HTTP requests, validates input, calls the appropriate service, returns a rendered page.	Flask routes
Face Enrollment & Recognition Service	Opens the webcam, detects faces (Haar cascade), captures/stores samples, trains and runs the LBPH recognizer.	OpenCV (opencv-contrib-python)
Attendance Service	Marks students present based on recognition results; supports faculty override.	Python / SQLite
Analytics & Report Service	Computes attendance percentages and generates CSV exports.	Python, pandas-style aggregation
Auth/Role Manager	Bootstraps the first Admin on first run (self-service /setup, no seed accounts), lets an existing Admin authorize new Admin/Faculty accounts (/register), authenticates logins (checking enrollment status for Students), and restricts routes/actions by role.	Flask session + password hashing
SQLite Database	Persists student records and attendance history.	SQLite3
Dataset/Trainer file store	Holds captured face images (dataset/) and the trained model file (trainer/trainer.yml).	Local filesystem
Webcam	Captures live video frames for enrollment and recognition.	OS camera driver via OpenCV VideoCapture

5. Data Flow Overview

5.1 Enrollment flow

Faculty/Admin submits the enroll form (name, roll number) -> Route Controller inserts a new Student row -> Face Enrollment Service opens the webcam, detects and saves ~30 face images under dataset/<id>_<name>/ -> the LBPH recognizer is retrained on the full dataset and saved to trainer/trainer.yml.

5.2 Attendance flow

Faculty submits ‘take attendance’ -> Face Recognition Service loads the trained model, opens the webcam, and for each detected face compares it against enrolled encodings -> the result is held for review rather than saved immediately -> faculty confirms or corrects it on a review screen -> the Route Controller writes the final attendance table rows (one per student per day) and records who confirmed them.

5.3 Authentication flow

On a brand-new install, every request is intercepted (before any route runs) and redirected to a one-time setup page until an Admin account exists; that page is then permanently locked. From then on, logins are checked against the Users table, with Students additionally re-checked against the current Student roster on every login (so removing a student also

revokes their access without deleting their account). New Admin/Faculty accounts can only be created by someone who already holds valid Admin credentials, submitted alongside the new account's details on the same request.

6. Deployment View

For this mini-project, the entire system (Flask app, SQLite database file, and dataset/trainer folders) runs as a single process on the machine that has the webcam attached - typically the presenting student's own laptop, started with `python app.py` and accessed at http://127.0.0.1:5000. There is no separate application server, load balancer, or remote database in this version. A future iteration could split the database onto a shared server (e.g. MySQL/PostgreSQL, already anticipated in SRS Section 2.4, Operating Environment) so multiple classrooms could record into one shared system, but that is explicitly out of scope here.

7. Technology Stack Summary

Layer	Technology
Frontend	HTML5, Bootstrap 5 (CDN)
Backend/Web framework	Flask 3.x (Python)
Face detection	OpenCV Haar Cascade (bundled locally as data/haarcascade_frontalface_default.xml)
Face recognition	OpenCV LBPH Face Recognizer (opencv-contrib-python)
Database	SQLite 3 (file-based, via Python's sqlite3 module)
Report export	Python csv module (CSV); Flask Response for file download

8. Design Principles Applied

- Modularity: enrollment, recognition, attendance-marking, analytics, and auth are kept as separate functions/modules rather than one large script, so any one piece can be modified or replaced independently (e.g. swapping LBPH for a deep-learning recognizer later).
- Maintainability: the recognition/matching logic is isolated from Flask route handlers (see SRS NFR 6.4), so the web layer does not need to change if the underlying recognition algorithm changes.
- Extensibility: the layered structure leaves room to add, for example, an SMS/email notification service (SRS Section 7, Other Requirements) as a new module without restructuring the existing layers.
- Debuggability: each service function (capture_face_samples, train_model, recognize_and_mark) can be tested and debugged independently of the Flask web server, by calling it directly from a Python shell.

9. Architectural Risks and Mitigations

Risk	Mitigation
The Flask dev server and OpenCV camera window run in the same single-threaded process; a long-running capture could block other requests.	Capture sessions are capped at 30 seconds (MAX_SESSION_SECONDS) and run as a direct, short-lived blocking call rather than a background job, which is acceptable for a single-user classroom demo.
LBPH has no built-in ‘unknown person’ class and may misclassify an unenrolled face as a near match.	A tunable confidence threshold plus mandatory faculty review/override (REQ-OV-1..3) keeps a human in the loop before attendance is finalised.

Risk	Mitigation
Face data is personal/biometric; committing it to a public GitHub repo would be a privacy issue.	Raw images and trained model files are excluded via .gitignore; only code and schema are versioned (see SRS Section 2.5).